

F24-165-D-RTSLConverter

Project Team

Muhammad Malik	21I-2688
Mohammad Abubakar Siddiq	21I-2742
Ahmad Usama	20I-2655

Session 2021-2025

Supervised by

Dr. Hassan Raza

Co-Supervised by

N/A



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

October, 2024

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Scope	2
1.3	Modules	2
1.3.1	Speech-to-Text (STT)	3
1.3.2	Text-to-Gesture Mapping	3
1.3.3	Animation	3
1.3.4	Backend Communication	3
1.3.5	Chrome Extension API	4
1.3.6	Dataset Research	4
1.3.7	Testing and Debugging	4
1.3.8	Deployment	4
1.4	User Classes and Characteristics	5
2	Project Requirements	7
2.1	Use-case and Event-Response Table	7
2.1.1	Use-case Diagram	7
2.1.2	Event-Response Table	9
2.2	Functional Requirements	10
2.2.1	Speech-to-Text (STT) Module	10
2.2.2	Text-to-Gesture Mapping Module	10
2.2.3	Animation Module	11
2.2.4	Backend Communication Module	11
2.2.5	Chrome Extension API Module	11
2.2.6	Dataset Research Module	12
2.2.7	Testing and Debugging Module	12
2.2.8	Deployment Module	12
2.3	Non-Functional Requirements	13
2.3.1	Reliability	13
2.3.2	Usability	13
2.3.3	Performance	14
2.3.4	Security	14

2.4	Domain Model	14
3	System Overview	17
3.1	Context	17
3.2	Functionality	18
3.3	Design	19
3.4	Architectural Design	19
3.4.1	Modular Program Structure	20
3.4.2	High-Level Overview of Collaboration	21
3.5	Design Models	23
3.6	Class Diagram	23
3.7	Activity Diagram	25
3.8	Class-level Sequence Diagram	26
3.9	State Transition Diagram	29
3.10	Data Design	31
3.10.1	Data Structures	31
3.10.2	Database Design	32
3.10.3	Data Flow	33
4	Conclusion	35
	References	37

List of Figures

2.1	Use-Case Diagram For SIGNFUSION Project	8
2.2	Domain Model for SIGNFUSION Project	15
3.1	Multi-tiered Client-Server Architecture for SIGNFUSION	22
3.2	Class Diagram for SIGNFUSION Project	23
3.3	Activity Diagram For SIGNFUSION Project	25
3.4	Class-Level Sequence Diagram for SIGNFUSION Project	27
3.5	State Transition Diagram for SIGNFUSION Project	29

List of Tables

1.1	User Classes and Characteristics	5
2.1	Event-Response Table for SIGNFUSION Project	9

Chapter 1

Introduction

Our project develops a software solution that translates speech into animated sign language in real time. It overlays sleek, translucent animations on screen with low latency and high accuracy, using advanced machine learning. [?].

1.1 Problem Statement

[6] The increasing volume of video content on platforms like YouTube and other online streaming platforms presents an accessibility challenge for the deaf and hearing-impaired community. Despite big leaps in transcription technologies, there remains a lack of an inclusive experience for users who rely on ASL as their mode of communication. This lack of accessibility prevents a significant portion of the population from fully participating in educational, informative and entertainment media consumption. This further expands the gap in their accessibility to digital media and inclusion. Without a proper solution to this problem the deaf and hearing-impaired community will keep facing hardships trying to access opportunities and information that is widely available to others in the digital world[8].

1.2 Scope

The scope of this project is to develop a real-time software solution, SIGNFUSION, which will enable the conversion of speech and text into animated sign language for multimedia content. This system is designed to provide accessibility for the deaf and hearing-impaired community, particularly in relation to video content on platforms like YouTube. The primary functionality of SIGNFUSION is to overlay animated sign language on top of videos, ensuring that users can access and engage with the content without relying on subtitles alone.

One of the key features of this project is its ability to perform real-time speech-to-sign translation, which translates spoken language into animated American Sign Language (ASL) gestures with low latency and high accuracy. Additionally, the system will allow text-to-sign translation for scenarios where pre-existing subtitles or text are available. Another important aspect is the customization of the avatar that performs the sign language gestures, providing users with the ability to personalize their experience.

The project will be delivered as a Chrome extension, making it easy for users to install and use on YouTube or other supported platforms. By integrating with Chrome's APIs, the extension will capture audio from videos, process the speech or text, and map it to corresponding ASL gestures in real time. The system will focus on minimizing latency and ensuring smooth, high-quality animations that align with the spoken words.

This solution is intended to bridge the accessibility gap for the deaf and hearing-impaired, allowing them to consume digital content in a more natural and engaging way. The scope of the project also includes testing and optimization of the system across different environments to ensure consistent performance. The goal is to create a scalable and customizable platform that can be expanded to other video platforms in the future.

1.3 Modules

The project is composed of several key modules that enable the real-time translation of sign language through a Chrome extension. Each module plays a critical role in ensuring the system functions efficiently, delivering accurate sign language translation with minimal latency.

1.3.1 Speech-to-Text (STT)

[1] This module is responsible for converting spoken language from video content into text. It is essential for enabling the translation of audio input into sign language.

1. Converts speech from video into text in real-time.
2. Processes audio input from videos, supporting a variety of speech patterns.
3. Ensures accurate transcription of speech to text for further processing.

1.3.2 Text-to-Gesture Mapping

[5] This module handles the mapping of recognized text to corresponding American Sign Language (ASL) gestures. It ensures the translation of spoken or written language into sign language is accurate and comprehensive.

1. Maps the recognized text to ASL gestures.
2. Ensures high accuracy in sign language translation.
3. Supports a large number of ASL signs for a wide variety of vocabulary.

1.3.3 Animation

[9] The animation module is responsible for generating and rendering sign language animations in near real-time. It ensures that the translated gestures are visually accurate and synchronized with the speech or text.

1. Generates animations in near real-time for smooth transitions.
2. Renders sign language animations in synchronization with video content.

1.3.4 Backend Communication

This module handles the communication between the Chrome extension and the backend server. It ensures that data is efficiently transmitted, processed, and stored.

1. Manages data requests between the frontend (Chrome extension) and the backend server.
2. Handles data retrieval and storage for sign language gestures and animations.
3. Ensures efficient data handling and low-latency communication.

1.3.5 Chrome Extension API

This module integrates the Chrome extension API, allowing the system to capture and process audio input from videos. It serves as the core interface for interaction between the extension and the video content.

1. Integrates the Chrome extension API for video interaction.
2. Allows the extension to capture audio input from videos.
3. Processes audio input in real-time for translation into sign language.

1.3.6 Dataset Research

[3] This module is focused on the collection, preprocessing, and management of sign language datasets. It ensures that the system has access to a comprehensive and accurate set of ASL gestures.

1. Collects and curates datasets of ASL gestures.
2. Preprocesses the dataset to ensure compatibility with the animation and gesture mapping modules.
3. Continuously improves the dataset for enhanced translation accuracy.

1.3.7 Testing and Debugging

This module ensures the system is fully tested and debugged prior to deployment. It focuses on identifying and resolving issues to ensure smooth operation across different scenarios.

1. Tests the Chrome extension in various environments to ensure reliability.
2. Identifies and resolves issues related to translation and animation.

1.3.8 Deployment

This module is responsible for packaging and deploying the Chrome extension to end users. It ensures that the setup and installation process is smooth and user-friendly.

1. Packages the Chrome extension for deployment.
2. Ensures a smooth and seamless setup process for end users.

1.4 User Classes and Characteristics

The SIGNFUSION project is designed to cater to a variety of users, each with unique roles and characteristics. These user classes have distinct interaction patterns with the system, ranging from casual usage to administrative functions. Below, the primary user classes and their respective characteristics are outlined.

User Class	Description
General User	General users represent individuals who are deaf or hearing-impaired, and they rely on the SIGNFUSION Chrome extension to access multimedia content, particularly video platforms like YouTube. They will use the extension to view sign language translations of the spoken content in real-time. This class typically does not require technical expertise and expects seamless integration with video content.
Content Creators	Content creators are individuals who produce video content for platforms like YouTube and want to ensure their content is accessible to a broader audience, including those with hearing impairments. They may test their videos using SIGNFUSION to verify the accuracy of the sign language translation, and potentially provide manual subtitle inputs. They require basic technical knowledge to operate the extension.
Administrators	Administrators manage the backend system, ensuring the accuracy and efficiency of the sign language translation process. Their responsibilities include updating the gesture dataset, managing system performance, and troubleshooting issues. They need advanced technical expertise to handle dataset updates, monitor performance metrics, and maintain system functionality.
Testers	Testers are responsible for ensuring the reliability and functionality of SIGNFUSION across different environments. Their tasks include identifying potential issues in sign language translation, synchronization, and animation. They use the system in different scenarios to identify bugs and ensure the extension is ready for deployment. Testers require technical understanding of both the frontend and backend systems.

Table 1.1: User Classes and Characteristics

Chapter 2

Project Requirements

This chapter describes the functional and non-functional requirements of the SIGNFUSION project. SIGNFUSION is a Chrome extension aimed at translating speech and text into animated sign language for video content, particularly on platforms like YouTube. The system must meet various functional requirements to ensure it operates effectively and provides accurate, real-time sign language translations. Additionally, non-functional requirements, such as performance, usability, and security, are essential to ensuring the system performs reliably and meets user expectations.

2.1 Use-case and Event-Response Table

The SIGNFUSION system addresses the accessibility needs of the hearing-impaired community by enabling real-time translation of video content into American Sign Language (ASL). The system functions as a real-time platform where most of the operations are handled in the backend to ensure efficient, low-latency sign language translation. Below, we present a use-case diagram followed by an event-response table.

2.1.1 Use-case Diagram

The use-case diagram below illustrates how different user classes (General User, Administrator, and Content Creator) interact with the SIGNFUSION system and perform key actions, such as watching videos with ASL translation, customizing avatars, and updating the gesture dataset.

The diagram above shows interactions between the system's user classes (General User, Administrator, Content Creator) and the various actions or functionalities they can perform.

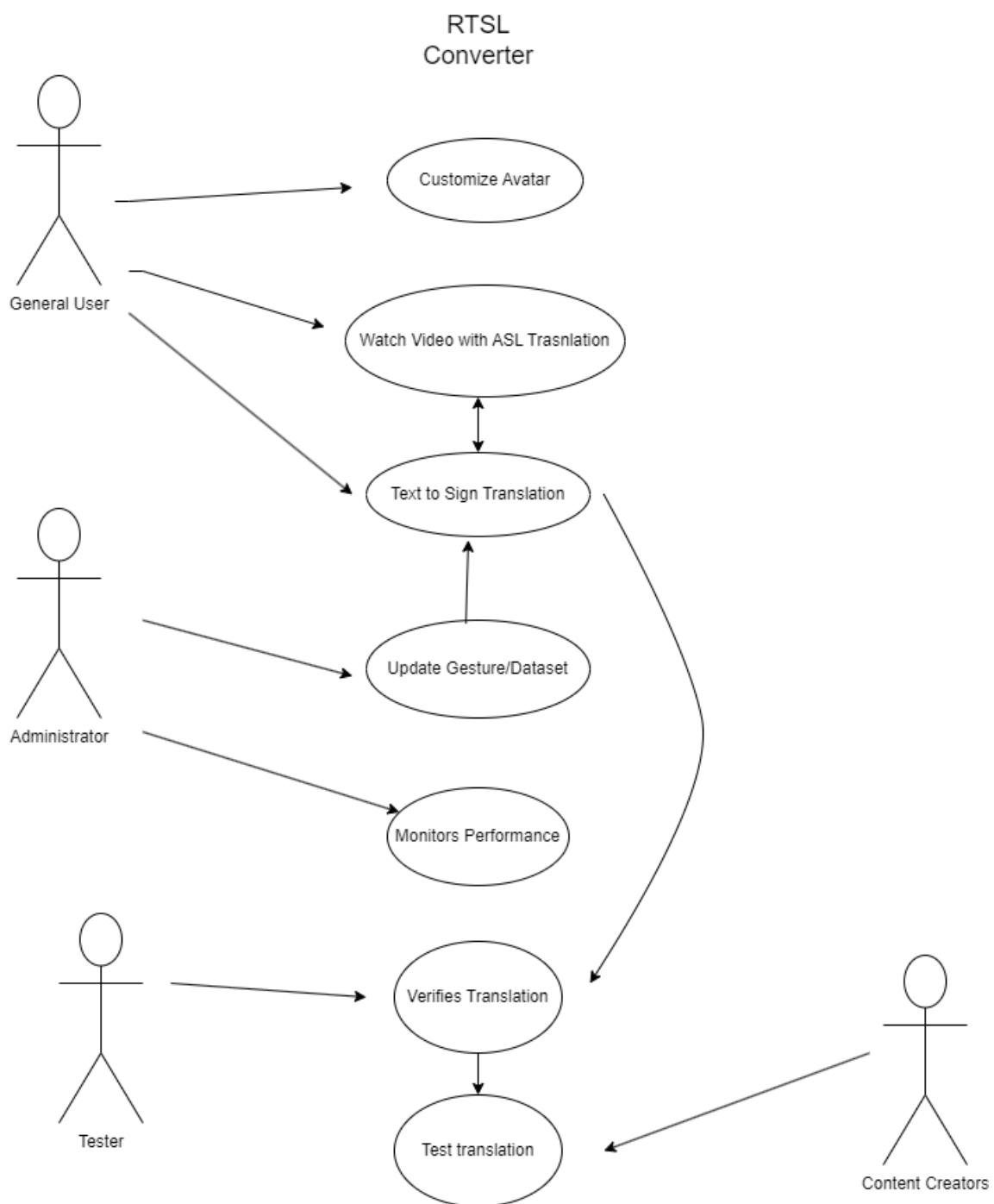


Figure 2.1: Use-Case Diagram For SIGNFUSION Project

2.1.2 Event-Response Table

The event-response table below describes various events that occur within the SIGNFUSION system and their corresponding backend system responses. These events help ensure real-time processing, low-latency translation, and accuracy in the sign language rendering.

Event	System Response
User starts a video on YouTube	The system captures the video's audio stream and sends it to the backend for real-time speech-to-text processing.
Speech-to-Text (STT) conversion starts	The backend processes the audio, converts speech to text, and sends the transcription to the gesture mapping module.
Text transcription received by backend	The system maps the text to corresponding ASL gestures using the gesture mapping module, retrieving appropriate signs from the ASL dataset.
ASL gestures mapped	The system generates real-time sign language animations based on the mapped gestures and syncs them with the video.
Animation rendered	The system overlays the sign language animation on top of the video, displaying it to the user in near real-time.
User customizes animation box	The user customizes the size and position of the animation box. The system adjusts the display according to the user's preferences and re-renders the animation.
Administrator updates ASL dataset	The backend receives new or updated ASL gestures and incorporates them into the gesture mapping module for more accurate translation.
System detects failure in translation	The system logs the error and triggers a fallback mechanism to retry or restore the last successful translation.

Table 2.1: Event-Response Table for SIGNFUSION Project

2.2 Functional Requirements

The functional requirements for SIGNFUSION describe the essential operations of the system, including real-time transcription, gesture mapping, animation, and data handling. These requirements are categorized by module to ensure each component of the system functions cohesively.

2.2.1 Speech-to-Text (STT) Module

[1] The STT module is responsible for converting spoken language from videos into text, which is then used for sign language translation.

1. **FR1:** The system shall convert speech from video content into text in real-time.
2. **FR2:** The system shall support various speech patterns, accents, and tones to ensure accurate transcription.
3. **FR3:** The system shall handle background noise and filter out non-speech audio to improve transcription accuracy.

2.2.2 Text-to-Gesture Mapping Module

The Text-to-Gesture Mapping module maps recognized text to corresponding American Sign Language (ASL) gestures.

1. **FR1:** The system shall map recognized text to the appropriate ASL gestures with high accuracy.
2. **FR2:** The system shall support a comprehensive database of ASL signs for a wide variety of vocabulary.
3. **FR3:** The system shall ensure that sign language gestures align with the corresponding text and are contextually accurate.

2.2.3 Animation Module

[5] The animation module generates and renders the ASL gestures in real-time, ensuring smooth and accurate sign language interpretation.

1. **FR1:** The system shall generate sign language animations in near real-time to synchronize with the speech or text.
2. **FR2:** The system shall ensure animations are smooth and natural, avoiding any abrupt or unnatural transitions.
3. **FR3:** The system shall support avatar customization, allowing users to personalize the appearance of the sign language avatar.

2.2.4 Backend Communication Module

This module handles communication between the Chrome extension and the backend, ensuring that data is transmitted, processed, and stored efficiently.

1. **FR1:** The system shall facilitate real-time communication between the frontend (Chrome extension) and backend servers.
2. **FR2:** The system shall store and retrieve gesture data for accurate sign language animations.
3. **FR3:** The system shall ensure low-latency data transmission to maintain synchronization with video content.

2.2.5 Chrome Extension API Module

The Chrome Extension API module interacts with the Chrome browser to capture audio and video inputs.

1. **FR1:** The system shall capture audio input from video content using the Chrome Extension API.
2. **FR2:** The system shall process audio and video input in real-time for immediate sign language translation.
3. **FR3:** The system shall provide users with an interface to enable or disable the extension's features.

2.2.6 Dataset Research Module

[4] The Dataset Research module is responsible for managing the ASL gesture datasets that power the sign language translation system.

1. **FR1:** The system shall collect and maintain a dataset of ASL gestures to support accurate sign language translation.
2. **FR2:** The system shall preprocess the ASL dataset to ensure compatibility with gesture mapping and animation modules.
3. **FR3:** The system shall continuously update and improve the dataset to enhance translation accuracy.

2.2.7 Testing and Debugging Module

This module ensures that the system is rigorously tested and debugged to resolve any issues before deployment.

1. **FR1:** The system shall be tested in multiple environments to ensure consistent performance and reliability.
2. **FR2:** The system shall identify and resolve issues related to transcription, gesture mapping, and animation.

2.2.8 Deployment Module

The Deployment module ensures that the Chrome extension is packaged and made available for public use.

1. **FR1:** The system shall package the Chrome extension for easy installation and use by end users.
2. **FR2:** The system shall provide an intuitive setup and configuration process to facilitate user onboarding.

2.3 Non-Functional Requirements

The non-functional requirements describe the system's performance, usability, and other qualities that affect user experience and system reliability. These requirements ensure that SIGNFUSION operates smoothly and meets expectations for performance and security.

2.3.1 Reliability

The system must maintain high availability and low failure rates to provide continuous, real-time translation for users.

- The system shall have a Mean Time Between Failures (MTBF) of at least 30 days.
- In case of failure, the system shall recover within 5 minutes to minimize disruption.
- The system shall log all failures and performance issues for future debugging and improvement.

2.3.2 Usability

The usability of SIGNFUSION is critical, as it is intended for both casual users and content creators.

- The user interface of the Chrome extension shall be simple and intuitive, allowing users to enable or disable features with a single click.
- The system shall allow users to customize the avatar for sign language gestures to enhance personal engagement.
- The system shall offer clear instructions and error messages to guide users in resolving common issues.

2.3.3 Performance

SIGNFUSION must perform efficiently to ensure that sign language translations are delivered in real time without noticeable delays.

- The system shall process speech-to-text and gesture mapping within 500 milliseconds to ensure low-latency translation.
- The system shall support simultaneous translation of up to 5 video streams without performance degradation.
- 95% of sign language animations shall be rendered without noticeable lag or delays.

2.3.4 Security

Security is essential to protect users' data and ensure that the system is not vulnerable to malicious attacks.

- The system shall ensure that no personal user data is collected or stored during the translation process.
- The system shall prevent unauthorized access to the backend data storage or manipulation of sign language gestures.
- The system shall encrypt all communication between the Chrome extension and backend servers to prevent data interception.

2.4 Domain Model

[2] The domain model for SIGNFUSION represents the core components involved in the sign language translation process, including speech input, gesture mapping, animation rendering, and user interaction with the Chrome extension. It also highlights the relationship between the frontend, backend, and external resources like ASL datasets.

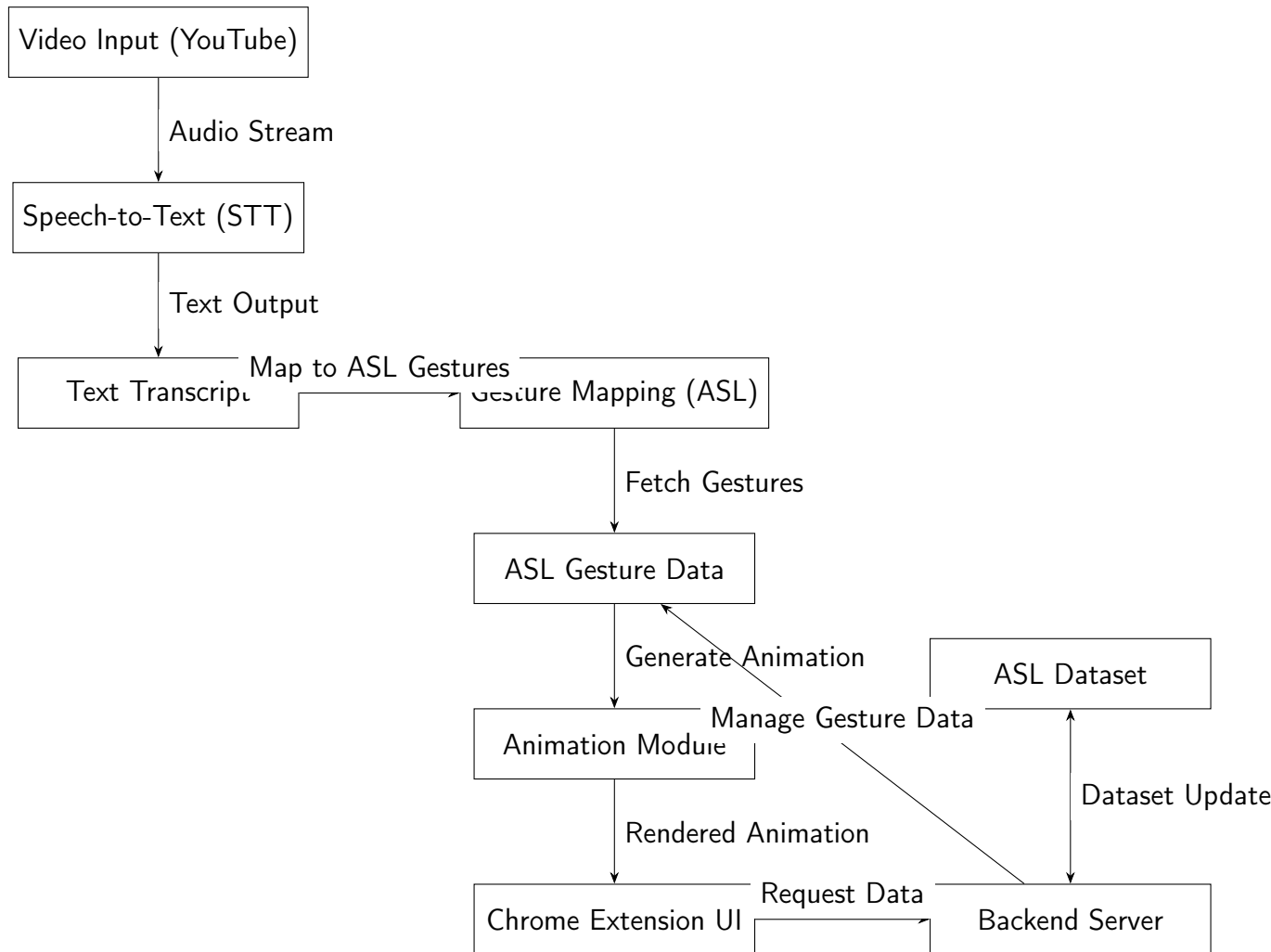


Figure 2.2: Domain Model for SIGNFUSION Project

Chapter 3

System Overview

The SIGNFUSION system is designed to provide a real-time translation of speech and text into animated sign language, specifically tailored for platforms like YouTube. The system is built to address the accessibility gap for hearing-impaired individuals by allowing them to consume multimedia content through American Sign Language (ASL) animations overlaid onto video content. SIGNFUSION seamlessly integrates with YouTube and operates via a Chrome extension, providing an intuitive interface for users to enable real-time ASL translation as they watch videos.

The key functionalities of SIGNFUSION include converting spoken words from video content into text through a Speech-to-Text (STT) engine, mapping the text to corresponding ASL gestures, and rendering these gestures as animations in real-time. The system ensures minimal latency between the spoken content and its translation into ASL, allowing users to experience synchronized ASL animations as they consume multimedia content.

In addition to real-time translation, the system offers a range of customization options, such as adjusting the size and position of the animated avatar on the screen. The system also allows content creators to test the accuracy of ASL translations for their videos and administrators to update the gesture datasets, ensuring accuracy and comprehensiveness over time.

3.1 Context

The hearing-impaired community faces significant challenges when accessing video content online, particularly on platforms like YouTube, which rely heavily on audio. While subtitles and closed captions offer some support, they fall short in catering to individuals who primarily use ASL as their mode of communication. SIGNFUSION aims to bridge this gap by providing a real-time translation tool that offers ASL translations directly over

video content, enabling hearing-impaired users to consume digital content more naturally and effectively.

The system is designed to cater to different user classes, including:

- **General Users:** Hearing-impaired individuals who use the Chrome extension to view real-time ASL translations of video content.
- **Content Creators:** Creators who wish to ensure their content is accessible to hearing-impaired users by testing and verifying the accuracy of ASL translations for their videos.
- **Administrators:** Users responsible for maintaining and updating the ASL gesture dataset, ensuring the system's accuracy and comprehensiveness over time.

SIGNFUSION integrates with Chrome and leverages various machine learning models and databases to achieve its functionality. By capturing audio input from videos and processing it through a robust STT engine, the system ensures accurate and real-time text extraction. The text is then mapped to ASL gestures using a pre-trained gesture mapping model, and the gestures are animated in near real-time on the screen, ensuring a smooth and synchronized experience for the user.

3.2 Functionality

The system's core functionality revolves around converting audio and text into visual ASL gestures. Below is a summary of the major processes that take place within the system:

1. **Speech-to-Text Conversion:** Audio from video content is processed by the STT module, which converts the speech into text in real-time.
2. **Text-to-Gesture Mapping:** The text output is passed through a gesture mapping module, which identifies corresponding ASL gestures for each recognized word or phrase.
3. **Rendering Animation:** The ASL gestures are rendered as animations overlaid onto the video, synchronized with the original spoken words.
4. **User Customization:** Users can customize the size and position of the animated avatar on their screen, ensuring a personalized viewing experience.

3.3 Design

SIGNFUSION is designed using a modular architecture, where each subsystem operates independently while contributing to the overall functionality. These subsystems include:

- **Speech-to-Text (STT) Module:** Handles audio input and converts it into text using advanced speech recognition algorithms.
- **Gesture Mapping Module:** Maps the recognized text to a corresponding ASL gesture.
- **Animation Module:** Renders the ASL gestures in real-time and displays them as smooth animations on top of the video.
- **Backend Communication Module:** Facilitates real-time communication between the Chrome extension and the backend server for data retrieval and storage.
- **Chrome Extension API:** Manages the interaction between the Chrome browser and the SIGNFUSION system, capturing audio input from videos.

The modular architecture ensures scalability and maintainability, allowing each component to be updated or improved independently without affecting the overall system. This design also ensures low-latency performance, making it suitable for real-time applications where synchronization between the spoken content and ASL animations is crucial.

Overall, the SIGNFUSION system offers a robust, scalable solution for translating video content into ASL in real-time. Through the use of advanced speech recognition, gesture mapping, and animation rendering technologies, the system empowers hearing-impaired individuals to engage with digital media content more naturally. The system's modular design and real-time performance make it an invaluable tool for improving accessibility in the digital world.

3.4 Architectural Design

The architecture of SIGNFUSION is designed to achieve real-time translation of speech and text into animated American Sign Language (ASL) gestures. This modular and layered architecture ensures that each subsystem operates independently while interacting with other components to deliver a seamless experience for users, particularly those accessing video content on platforms like YouTube. The system is built around key functional modules that collaborate to perform speech recognition, gesture mapping, animation rendering, and data communication efficiently.

3.4.1 Modular Program Structure

The system's architecture is divided into the following core modules, each responsible for distinct functionality:

- **Speech-to-Text (STT) Module:** This module is responsible for capturing the audio from video content and converting it into text. It uses a combination of Chrome's Web Speech API and Google's speech recognition technologies. The Web Speech API allows the system to extract spoken language from the video, while Google's automatic speech recognition (ASR) processes the audio input in real-time, transforming it into accurate text output. The STT module employs techniques such as noise filtering and normalization to handle various speech patterns, accents, and background noise, ensuring that the transcription remains clear and accurate.
- **Text-to-Gesture Mapping Module:** The text generated by the STT module is then passed to the Text-to-Gesture Mapping module. This component maps recognized words and phrases to their corresponding ASL gestures using a pre-built dataset of gestures. The system relies on a sophisticated machine learning model, trained with a large dataset of ASL signs, to ensure the text is contextually mapped to appropriate ASL signs. For instance, the module accounts for different meanings of the same word in various contexts, ensuring the right gesture is used based on the surrounding words. The system also uses techniques such as Natural Language Processing (NLP) to ensure the translated text maintains its semantic meaning when mapped to gestures.
- **Animation Module:** Once the ASL gestures are mapped, this module is responsible for rendering them as animations that are overlaid on the video. The module uses a custom animation engine that retrieves pre-animated ASL signs and seamlessly synchronizes them with the speech or text content of the video. The system ensures that the avatar used for these animations performs the signs with smooth transitions and lifelike motion, closely following the timing of the spoken words. The animation module also allows for customization, enabling users to adjust the appearance of the avatar and the position/size of the animation window on the video screen.
- **Backend Communication Module:** The Backend Communication module handles the interaction between the Chrome extension on the client-side and the backend server. The backend is responsible for managing the storage of ASL gesture data, processing user preferences, and supporting real-time data communication. This module ensures that the Chrome extension can fetch the necessary gesture data from the backend without significant latency. It also manages any custom configu-

rations set by the users, such as avatar personalization or gesture speed adjustments, and stores them for future sessions.

- **Chrome Extension API Module:** This module is the core interface between the user and the system. It interacts with the Chrome browser to capture audio and video streams from platforms like YouTube. The Chrome Extension API provides access to the video's audio feed, allowing the system to capture and process speech input. It also provides users with controls for enabling or disabling the SIGNFUSION functionality, adjusting the position of the animated ASL gestures, and modifying the avatar's appearance. By integrating with Chrome's APIs, this module ensures smooth interaction with the browser environment.
- **Dataset Research and Management Module:** This module is focused on managing the dataset of ASL gestures that powers the system. It involves the collection, preprocessing, and management of a large repository of ASL signs. The dataset is continuously updated to include new gestures, improve existing ones, and expand vocabulary coverage. The module ensures that the system has access to accurate and up-to-date gesture data for mapping text to ASL.
- **Testing and Debugging Module:** The Testing and Debugging module is essential for ensuring the system operates smoothly across various environments and video platforms. The module automates tests to validate the accuracy of speech recognition, gesture mapping, and animation rendering. It also provides tools for developers to debug any issues related to synchronization, latency, or misinterpretation of gestures.

3.4.2 High-Level Overview of Collaboration

Each module in SIGNFUSION interacts with the others to form a cohesive system that delivers accurate, real-time ASL translations.

The architecture includes the following layers:

- **Presentation Layer:** The Chrome extension serves as the presentation layer, where users interact with the system.
- **Application Layer:** This layer handles the logic of the application, including speech-to-text conversion, gesture mapping, and animation rendering.
- **Data Layer:** The data layer stores gesture datasets and user preferences, managed through the backend communication module.

The multi-tiered design ensures that each layer focuses on a specific function, enhancing the system's performance, scalability, and maintainability.

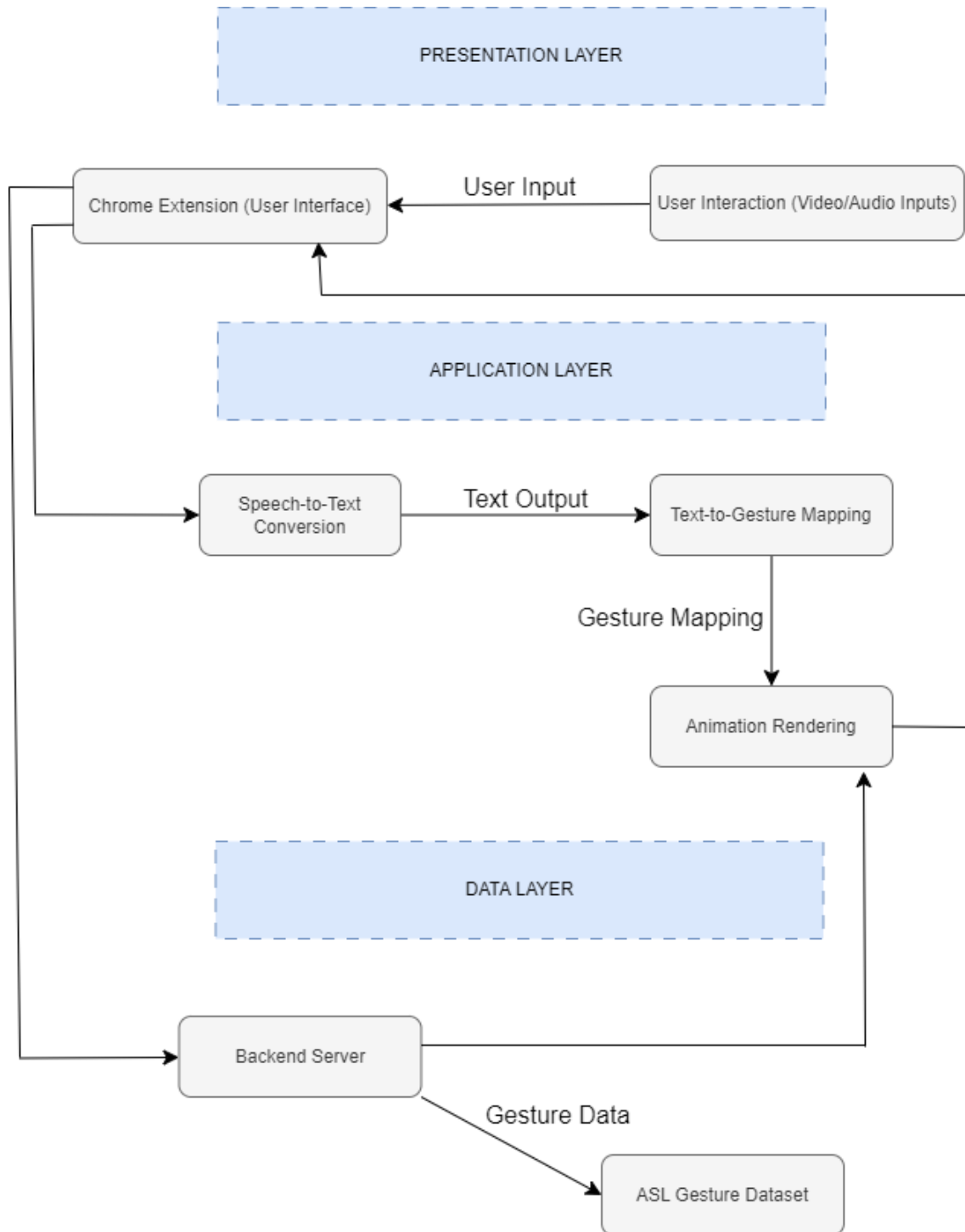


Figure 3.1: Multi-tiered Client-Server Architecture for SIGNFUSION

3.5 Design Models

[7] This provides detailed design models for the SIGNFUSION project, focusing on the structural and behavioral aspects of the system. These models give a comprehensive view of how the system's components interact and function.

3.6 Class Diagram

The class diagram for SIGNFUSION showcases the various classes and their relationships, defining the structural design of the system and the interactions between its core components. This diagram breaks down the functionality of the system into separate entities, each responsible for a distinct role, ensuring modularity and maintainability.

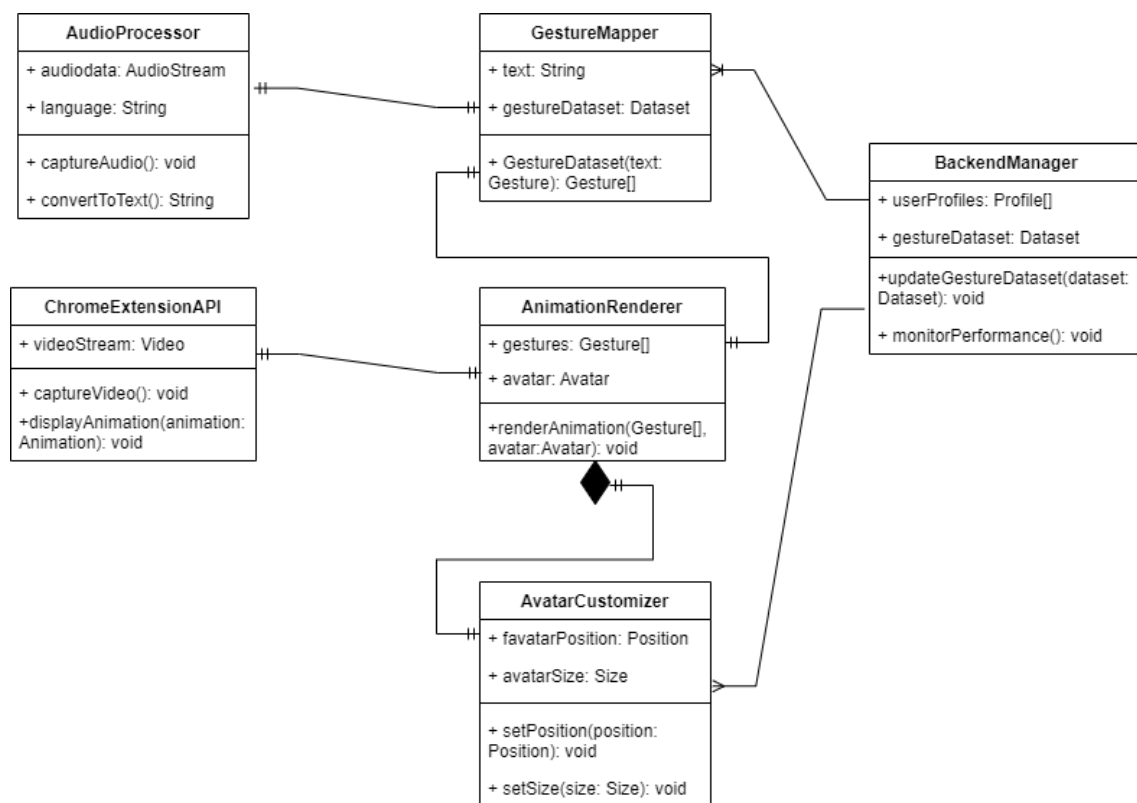


Figure 3.2: Class Diagram for SIGNFUSION Project

- **AudioProcessor:** This class is responsible for capturing audio from video streams (e.g., YouTube) and converting it into text. It has methods for capturing the audio data and processing it into transcriptions, such as:
 - `captureAudio()`: Captures the audio stream for further processing.
 - `convertToText()`: Converts the captured audio stream into text for gesture mapping.
- **GestureMapper:** The `GestureMapper` class maps the converted text to the respective ASL gestures. It ensures the correctness of the gesture mapping by cross-referencing with the dataset. Its key methods include:
 - `GestureDataset()`: Retrieves the corresponding ASL gestures based on the input text.
- **ChromeExtensionAPI:** This class is responsible for interfacing with the Chrome browser, allowing the system to capture video streams, process them, and display animations. Its methods include:
 - `captureVideo()`: Captures the video stream that needs ASL translation.
 - `displayAnimation()`: Displays the generated sign language animation on the screen.
- **AnimationRenderer:** The `AnimationRenderer` is tasked with generating and rendering the ASL gestures in real-time. This class creates animations based on the gestures provided by the `GestureMapper`. Key methods:
 - `renderAnimation()`: Renders the animation using the appropriate avatar and gestures.
- **AvatarCustomizer:** This class allows users to customize the avatar's size and position for better visibility and personalization of the sign language animation. It contains methods such as:
 - `setPosition()`: Sets the position of the sign language avatar on the screen.
 - `setSize()`: Adjusts the avatar's size based on user preferences.
- **BackendManager:** The `BackendManager` handles the backend processes, such as managing user profiles, updating the gesture dataset, and monitoring system performance. It ensures that data flows between the frontend (Chrome Extension) and the backend server are efficient and secure. Methods include:
 - `updateGestureDataset()`: Updates the gesture dataset with new or improved gestures.
 - `monitorPerformance()`: Monitors the overall performance of the system.

3.7 Activity Diagram

The activity diagram for SIGNFUSION provides a detailed flow of how the system processes the video input and generates the corresponding ASL animation for the user. It outlines the sequential steps taken from the moment the user activates the Chrome extension until the final animation is displayed.

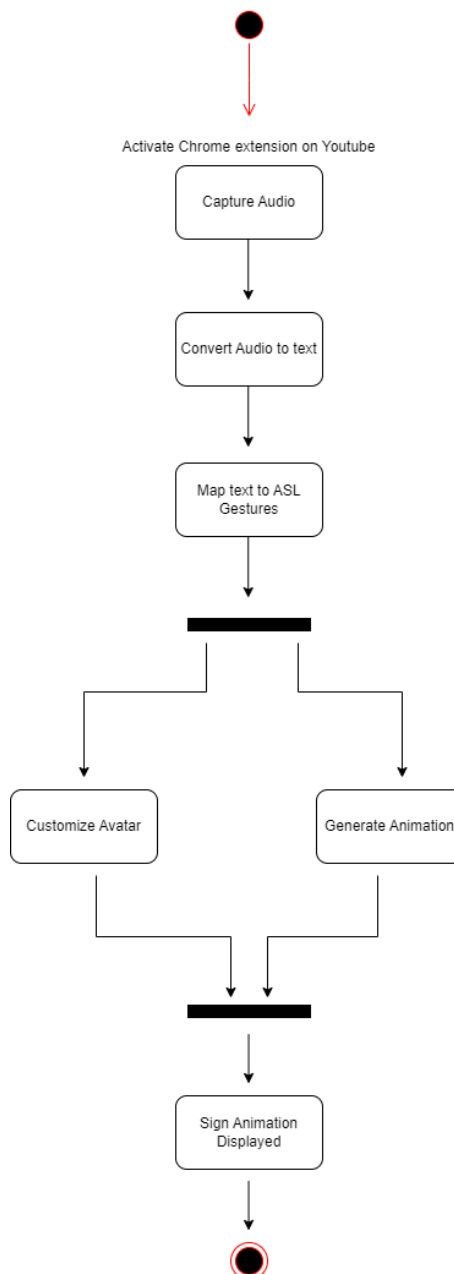


Figure 3.3: Activity Diagram For SIGNFUSION Project

- **Activate Chrome Extension:** The process begins when the user activates the SIGNFUSION Chrome extension on a YouTube video. This triggers the subsequent steps.
- **Capture Audio:** Once activated, the system captures the audio from the video stream. The captured audio is passed to the AudioProcessor for further processing.
- **Convert Audio to Text:** The AudioProcessor converts the captured audio stream into text using speech-to-text technologies. This step is crucial for enabling the gesture mapping.
- **Map Text to ASL Gestures:** The GestureMapper class takes the text and maps it to the corresponding ASL gestures by referencing the gesture dataset.
- **Customize Avatar:** In this step, the user has the option to customize the avatar's size and position, enhancing personalization and visibility for better sign language display.
- **Generate Animation:** Simultaneously, the AnimationRenderer generates the corresponding sign language animation based on the mapped ASL gestures.
- **Sign Animation Displayed:** Finally, the generated animation is displayed on the video content through the Chrome extension, allowing the user to view the translated sign language gestures in real-time.

3.8 Class-level Sequence Diagram

The sequence diagram for SIGNFUSION illustrates the interaction between various objects (or classes) over time. It provides a dynamic view of the system, showing how processes flow sequentially as the user interacts with the system. The sequence diagram demonstrates the steps involved in capturing audio from a video stream, mapping it to ASL gestures, and displaying the corresponding animations.

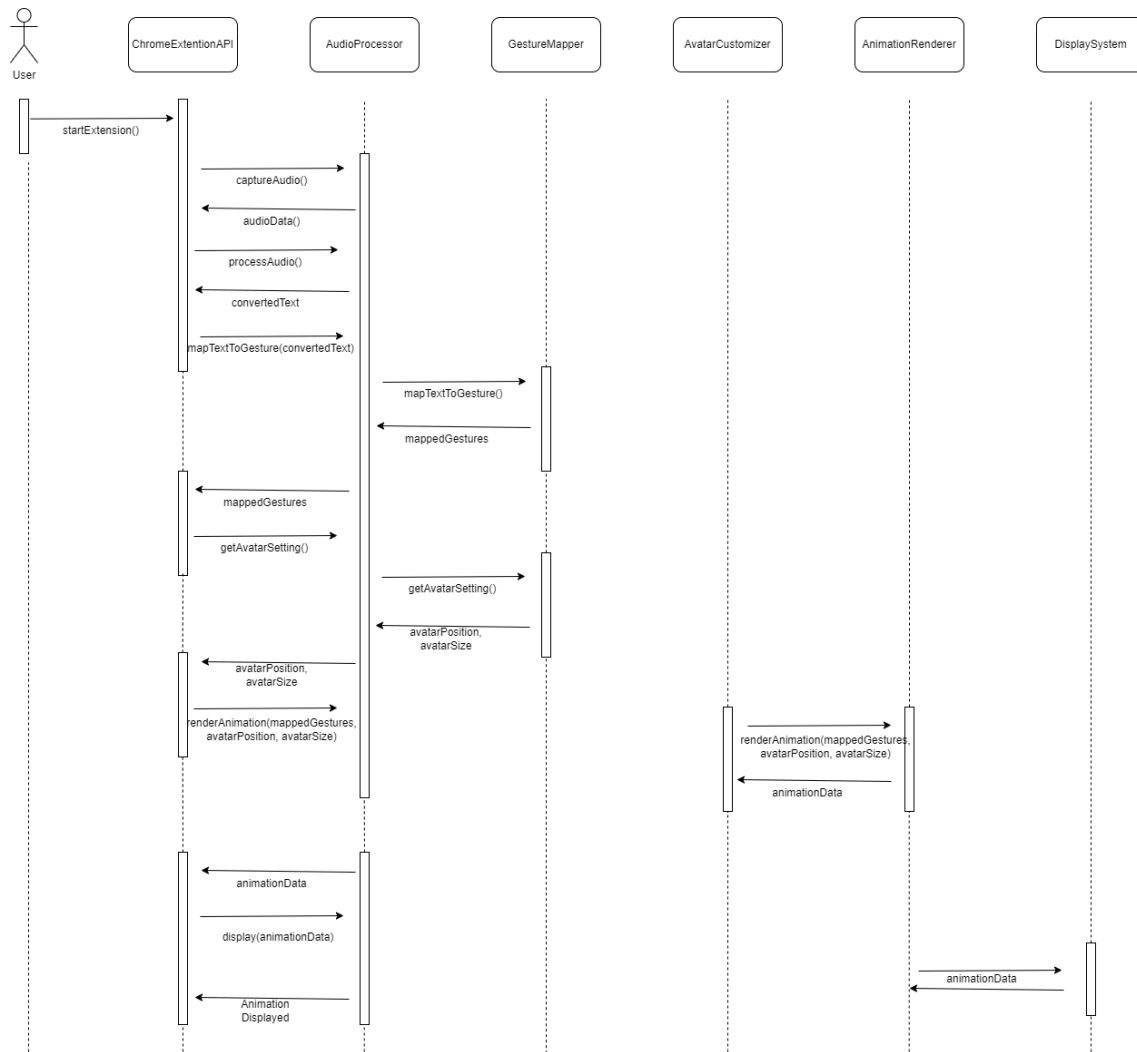


Figure 3.4: Class-Level Sequence Diagram for SIGNFUSION Project

Detailed Process

1. User Initiates Chrome Extension:

The user starts by activating the SIGNFUSION Chrome extension when watching a video. This triggers the sequence of events to capture the audio and display the sign language animation.

2. ChromeExtensionAPI (startExtension):

The Chrome extension captures video streams and processes them. In this step, it initiates the action to start capturing the audio from the video stream using the Chrome API.

3. **AudioProcessor (captureAudio, processAudio):**

The AudioProcessor is responsible for capturing and processing the audio stream. It converts the audio data into text, which is then passed to the next stage of processing for gesture mapping.

- *captureAudio()*: Captures the audio stream from the video.
- *processAudio()*: Processes the captured audio and converts it into text.

4. **GestureMapper (mapTextToGesture):**

After receiving the converted text from the AudioProcessor, the GestureMapper class maps the text to the corresponding ASL gestures. It uses the gesture dataset for this mapping.

- *mapTextToGesture()*: Maps the converted text to gestures.
- *mappedGestures*: A set of gestures corresponding to the input text.

5. **AvatarCustomizer (getAvatarSetting):**

The AvatarCustomizer retrieves the user's avatar settings, including position and size, which are used to display the avatar performing the gestures.

- *getAvatarSetting()*: Gets the user's avatar position and size settings.

6. **AnimationRenderer (renderAnimation):**

This component renders the animation by using the mapped gestures and the avatar settings. The final output is a set of animations representing the translated ASL gestures.

- *renderAnimation()*: Renders the sign language animations based on the gestures and avatar settings.

7. **DisplaySystem (display):**

Finally, the DisplaySystem takes the generated animation data and displays it on the video stream, allowing the user to see the real-time sign language translation.

This sequence diagram provides a step-by-step view of how the SIGNFUSION system processes audio input and displays the corresponding sign language animations. Each component (class) has a specific role in the workflow, ensuring a smooth and efficient user experience.

3.9 State Transition Diagram

The state transition diagram for SIGNFUSION represents the different states that the system undergoes during its operation, from when the user starts the Chrome extension to when the sign language animation is displayed and finally when the extension is stopped. Each state transition is triggered by an event, with specific methods invoked in each state to manage the process.



Figure 3.5: State Transition Diagram for SIGNFUSION Project

Detailed Process

1. Idle State:

Initially, the SIGNFUSION system remains idle, waiting for the user to start the extension.

- *startExtension()*: When the user activates the Chrome extension, the system transitions from the Idle state to the Audio Capture state.

2. Audio Capture State:

In this state, the system captures audio from the video stream.

- *audioCaptured()*: The audio data is captured and passed to the next state, where the speech-to-text conversion process occurs.

3. Speech-to-Text Conversion State:

The system converts the captured audio into text using the Speech-to-Text module.

- *textConverted()*: Once the speech is converted into text, the system transitions to the Gesture Mapping state.

4. Gesture Mapping State:

The converted text is mapped to ASL gestures using the Gesture Mapping module.

- *gestureMapped()*: The mapped gestures are then ready for animation generation or customization.

5. Avatar Customization State (Optional):

In this state, users can customize the avatar, specifically its position and size on the screen.

- *avatarCustomized()*: The system applies the user's customization settings and prepares for animation generation.

6. Animation Generation State:

The system generates the animation using the mapped gestures and avatar settings (if customized).

- *animationGenerated()*: Once the animation is generated, the system transitions to the Display Animation state.

7. Display Animation State:

The system displays the generated sign language animation over the video stream.

- *animationDisplayed()*: The sign language animation is shown to the user in sync with the video content.

8. Stop Extension State:

When the user decides to stop the extension, the system returns to the Idle state.

- *stopExtension()*: The system stops all ongoing processes and transitions back to the Idle state, awaiting the next activation.

This state transition diagram provides a comprehensive view of how the SIGNFUSION system transitions between various states, ensuring smooth real-time sign language translation and user interactions with the extension.

3.10 Data Design

The data design for SIGNFUSION involves transforming the key entities of the system into organized data structures, ensuring efficient storage, processing, and retrieval of data. The data can be stored either in a traditional SQL database or a cloud-based database, depending on the scalability and performance needs of the system.

3.10.1 Data Structures

The system primarily handles the following data structures:

- **Gesture Data:** The core data structure in SIGNFUSION is the set of American Sign Language (ASL) gestures, which are mapped to their corresponding text and speech inputs. Each gesture entry contains:
 - Gesture ID (Primary Key)
 - Gesture Name
 - Gesture Animation File
 - Corresponding Text and Speech Phrases
- **User Preferences:** For each user, the system stores customization settings such as avatar appearance, size, and position for the sign language animations. Each user entry includes:
 - User ID (Primary Key)
 - Avatar Settings (Position, Size)
 - Interaction Logs (For performance monitoring and personalization)

- **Video Interaction Data:** The system may also store interaction data related to the videos that have been processed for ASL translation, helping monitor system performance and user engagement. Each video entry contains:
 - Video ID
 - Video URL
 - Duration Processed
 - Timestamps of Sign Language Overlay

3.10.2 Database Design

The data mentioned above is organized into structured tables that can be stored either in an SQL database or in a scalable cloud-based database solution. This decision is based on the project's scalability and performance needs.

- **SQL Database:** The system can leverage relational databases like MySQL or PostgreSQL to store data in a structured format. SQL databases are optimal for organizing gesture mappings, user preferences, and video interaction data through the use of tables, relationships, and indexing.
 - **Tables:**
 - * GestureTable (Gesture ID, Gesture Name, Gesture Animation, Text Phrase, Speech Phrase)
 - * UserPreferencesTable (User ID, Avatar Settings, Interaction Logs)
 - * VideoInteractionTable (Video ID, URL, Duration, Timestamp)
 - **Relationships:** User data is linked to their preferences, and video data can be cross-referenced with gestures for performance tracking.
- **Cloud-Based Database:** For more scalable needs, the system can utilize cloud-based databases such as Amazon RDS or Google Cloud SQL. These services provide high availability, easy backups, and automatic scaling, ensuring the system can grow with increasing data volumes.
 - Cloud databases offer similar structure as SQL databases, but with enhanced accessibility, distributed storage, and automated scaling.

3.10.3 Data Flow

The system processes data in the following way:

- Speech-to-text conversions and gesture mappings are stored in the database during or after each translation process.
- User preferences, such as avatar customization, are stored and updated whenever a user interacts with the customization options.
- Video interaction data is captured during the user's engagement with the system and is stored for performance monitoring and potential optimizations.

In this way, the data structures ensure efficient processing and organization of both user data and gesture-related information. The flexibility of using either a SQL or cloud-based database allows the system to scale as the user base grows or new features are added.

Chapter 4

Conclusion

The SIGNFUSION project set out with the ambitious goal of bridging the accessibility gap for the deaf and hearing-impaired community by providing real-time American Sign Language (ASL) translations for video content. Through the development of a sophisticated Chrome extension, SIGNFUSION successfully addresses the challenge of making multimedia content more inclusive by overlaying animated ASL gestures on videos, particularly on platforms such as YouTube.

Throughout the course of this project, we explored and implemented cutting-edge technologies to ensure a seamless and accurate user experience. SIGNFUSION integrates real-time speech-to-text conversion, advanced gesture mapping, and a robust backend to deliver smooth and efficient ASL translations. The project was structured using a multi-tiered client-server architecture, which divides the system into key layers—presentation, application, and data—ensuring efficient communication between the Chrome extension and the backend. This design provides modularity, scalability, and maintainability, allowing the system to evolve and expand as needed.

Key components such as the Speech-to-Text (STT) module, Gesture Mapping module, and Backend Communication module were designed to operate efficiently in concert. These modules were tested rigorously to ensure accuracy and performance across different environments. The Avatar Customizer feature provides a personalized user experience, enhancing engagement by allowing users to adjust the size and position of the animated avatar, thereby ensuring an inclusive interface for all types of users.

At present, the animation generation is still under active development. Improving the accuracy of gesture mapping and achieving near-real-time animation rendering are key objectives of the ongoing work. Our current focus is on refining the gesture-to-animation pipeline to ensure a faster and more accurate real-time experience for users. Future updates to the system will enhance the synchronization between gesture mapping and animation, minimizing any delays and further improving the smoothness of the animated

ASL gestures. Achieving low-latency performance remains a critical goal as we continue optimizing the system.

Additionally, the project's backend is designed to handle gesture datasets and user preferences using either SQL or cloud-based database solutions, providing flexibility for different deployment scenarios. This structure ensures that SIGNFUSION can scale as more users adopt the system and as new gesture data or features are added.

The design models, including the class diagram, activity diagram, sequence diagram, and state transition diagram, provide a detailed understanding of the system's internal operations. These models illustrate how data flows through the system, how components interact, and how different states are managed during the ASL translation process.

The final result, once animation generation is fully integrated, will be a powerful, accessible, and scalable solution that not only helps the deaf and hearing-impaired community consume digital content more easily but also sets the foundation for future expansion to other video platforms. With the increasing demand for digital content accessibility, SIGNFUSION offers a meaningful contribution toward building a more inclusive internet.

Future work on this project will focus on completing the animation generation, optimizing the accuracy and latency of the translation process, and extending the functionality to cover other sign languages. Additionally, we will continue improving the dataset of gestures and look to integrate with additional video platforms beyond YouTube. These enhancements will ensure a truly real-time and immersive experience for users.

In conclusion, SIGNFUSION represents a significant step forward in multimedia accessibility, utilizing state-of-the-art technology to deliver real-time ASL translations in an intuitive and user-friendly format. It underscores the importance of creating inclusive digital environments and demonstrates the potential of technology in breaking down barriers for underrepresented communities.

Bibliography

- [1] N. Ashok, K. S. Aswin, Suresh S. Babu P. N, and S. K. Suresh. Speech to sign translator. *International Journal of Creative Research Thoughts (IJCRT)*, 10(5):864–874, 2022.
- [2] D. Brentari, B. Shi, G. Shakhnarovich, and K. Livescu. Open-domain sign language translation learned from online video. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 6365–6379, 2022.
- [3] L. Dongxu, L. Hongdong, and R. Baskoro. World level american sign language video, 2021.
- [4] A. Duarte, S. Palaskar, L. Ventura, D. Ghadiyaram, K. DeHaan, F. Metze, J. Torres, and X. Giro-i Nieto. How2sign: A large-scale multimodal dataset for continuous american sign language. In *Computer Vision Foundation*, pages 2736–2744, 2021.
- [5] Eleni Efthimiou, Stavroula-Evita Fotinea, Christian Vogler, Thomas Hanke, John Glauert, Richard Bowden, Annelies Braffort, Christophe Collet, Petros Maragos, and Jérémie Segouat. Sign language recognition, generation, and modelling: A research effort with applications in deaf communication. In *Universal Access in Human-Computer Interaction. Addressing Diversity*, volume 5614 of *Lecture Notes in Computer Science (LNCS)*, pages 21–30. Springer, Berlin, Heidelberg, 2009.
- [6] N. Hernandez. Hearing loss statistics 2024: More common than you might think, 2023.
- [7] Dongxu Li, Yipin Wong, Yun Zhao, Mohan Kankanhalli, and Hongdong Li. Transferring cross-domain knowledge for video sign language recognition. *arXiv preprint arXiv:2004.14874*, 2020.
- [8] World Population Review. Countries that use asl 2024, 2024.
- [9] M. Villa-Monedero, M. Gil-Martín, D. Sáez-Trigueros, A. Pomirski, and R. San-Segundo. Sign language dataset for automatic motion generation. *Journal of Imaging*, 9(12):1–12, 2023.